

CF-DETR: Coarse-to-Fine Transformer for Real-Time Object Detection

Woojin Shin^{1,*}, Donghwa Kang^{2,*}, Byeongyun Park¹, Brent Byunghoon Kang²,
Jinkyu Lee^{3,†} and Hyeongboo Baek^{1,†}

¹University of Seoul ²KAIST ³Yonsei University
hbbaek@uos.ac.kr

Abstract—Detection Transformers (DETR) are increasingly adopted in autonomous vehicle (AV) perception systems due to their superior accuracy over convolutional networks. However, concurrently executing multiple DETR tasks presents significant challenges in meeting firm real-time deadlines (R1) and high accuracy requirements (R2), particularly for safety-critical objects, while navigating the inherent latency-accuracy trade-off under resource constraints. Existing real-time DNN scheduling approaches often treat models generically, failing to leverage Transformer-specific properties for efficient resource allocation. To address these challenges, we propose **CF-DETR**, an integrated system featuring a novel coarse-to-fine Transformer architecture and a dedicated real-time scheduling framework **NFPF****. **CF-DETR** employs three key strategies (A1: coarse-to-fine inference, A2: selective fine inference, A3: multi-level batch inference) that exploit Transformer properties to dynamically adjust patch granularity and attention scope based on object criticality, aiming to satisfy R2. The **NFPF**** scheduling framework (A4) orchestrates these adaptive mechanisms A1–A3. It partitions each DETR task into a safety-critical coarse subtask for guaranteed critical object detection within its deadline (ensuring R1), and an optional fine subtask for enhanced overall accuracy (R2), while managing individual and batched execution. Our extensive evaluations on server, GPU-enabled embedded platforms, and actual AV platforms demonstrate that **CF-DETR**, under an **NFPF**** policy, successfully meets strict timing guarantees for critical operations and achieves significantly higher accuracy compared to existing baselines across diverse AV workloads.

I. INTRODUCTION

Detection Transformers (DETR) have significantly advanced object detection, outperforming traditional convolutional networks in accuracy and, in some instances, inference speed, making them highly attractive for autonomous vehicle (AV) perception systems [1], [2]. This superior performance stems from their global *attention* mechanism, which processes images as split *patch* collections to capture comprehensive contextual relationships [3]. However, this powerful mechanism also incurs substantial computational costs that scale markedly with input patch count, creating a pronounced latency-accuracy trade-off [4], [5]. For AVs concurrently running multiple DETR tasks (e.g., per camera) under stringent real-time deadlines (R1) and high accuracy demands (R2)—especially for safety-critical objects—this trade-off presents a

formidable challenge [6], [7]. The fundamental design goal is thus to satisfy both R1 and R2, prioritizing critical objects within typical AV resource constraints [7]–[9].

Since the advent of vision Transformers (ViTs) approximately five years ago, various lightweighting techniques have been explored. However, adaptively managing the computational load of such models specifically for multi-task DETR execution within safety-critical real-time systems remains a significant, largely unaddressed challenge. Prevailing real-time DNN scheduling studies [6], [7], [10]–[12] predominantly focus on convolutional architectures or treat DNNs as monolithic black boxes, lacking a nuanced understanding of internal Transformer properties like the critical impact of patch granularity and attention scope on the latency-accuracy trade-off. Among recent lightweight ViT methods [4], [5], coarse-to-fine (CF) strategies are gaining traction in classification (e.g., CF-ViT [13]); however, their systematic application to *detection* transformers is a largely unaddressed area. This is primarily due to detection’s sophisticated spatial understanding requirements and the inherent challenge of integrating fine-grained adaptability with real-time scheduling. Our key insight, drawn from experimental observations (Sec. II-B), is that a CF approach tailored for DETR (*first* proposed in this paper) not only effectively navigates the latency-accuracy trade-off but also excels in prioritizing responsiveness and accuracy for safety-critical objects under constrained resources.

In this paper, we propose **CF-DETR**, an integrated system featuring a novel coarse-to-fine DETR architecture coupled with a dedicated real-time scheduling framework. The **CF-DETR** architecture primarily tackles the challenge by leveraging fundamental Transformer properties to navigate the latency-accuracy trade-off while considering object criticality. **CF-DETR** proposes three complementary approaches A1–A3 (to be presented) that optimize DETR’s costly self-attention stage. As the computational complexity of self-attention scales with the number of patches, there is an explicit trade-off between fewer patches, resulting in faster inference but potentially detecting only larger critical objects, and more patches, yielding slower but more accurate detection for objects of all sizes. **CF-DETR** strategically leverages this property by dynamically adjusting patch granularity and attention scope based on each input’s object criticality.

A1. Coarse-to-fine inference (image-level allocation): Some images are *easy* to achieve high accuracy on, achieving

*Equally contributed.

†Hyeongboo Baek is the corresponding author. Jinkyu Lee (co-corresponding author) initiated this work while affiliated with Sungkyunkwan University.

sufficient accuracy even with a coarse patch division (e.g., 3×3), typically containing only large, critical objects. Conversely, others are *hard*, containing objects of varied sizes, which require finer patch divisions (e.g., 9×9) for accurate detection. In A1, CF-DETR first processes each image rapidly with a coarse-grained DETR inference. It then identifies images exhibiting low-confidence detections (*hard* images) for further refinement at a finer granularity in A2.

- A2. *Selective fine inference* (region-level allocation): Even within a hard image, not all regions are equally challenging—many parts containing large and critical objects of the image can be handled with coarse patches, while only certain regions need finer detail. Building on A1, A2 identifies and refines only the hard regions within hard images using more patches, thus avoiding full image reprocessing and dedicating computation efficiently to the areas that benefit most.
- A3. *Multi-level batch inference* (batch-level allocation): Effective batching is crucial for throughput in AV systems running multiple concurrent DNN tasks, such as processing data from multiple cameras. A3 optimizes batch processing through a multi-level approach: for initial coarse-grained inference A1, CF-DETR performs *image-level* batch execution across multiple input frames. Subsequently, leveraging outputs from A1 and A2, it executes *patch-level* batch inference exclusively on consolidated fine-grained hard regions from multiple tasks.

To manage these adaptive CF-DETR mechanisms A1–A3 across multiple DETR instances and effectively address the multi-task coordination challenge, we introduce (A4) the NPFP** (Non-Preemptive Fixed-Priority with generic coarse-to-fine policy “**”) scheduling framework. This framework orchestrates the CF-DETR architecture by partitioning each DETR task into two key components: a safety-critical coarse subtask (primarily executing A1) and an optional, adaptive fine subtask (mainly executing A2). While the timely completion of coarse subtasks (without batching) is guaranteed by assigning higher priority to coarse subtasks over fine subtasks, the framework’s flexibility allows different execution modes for coarse and fine subtasks, each with either individual or batched processing. To effectively integrate batching for improved overall accuracy without compromising timing guarantees, we develop novel batch assignments for each of the coarse and fine batches, which not only minimize the run-time overhead but also utilize the properties of batching and DETR.

Our extensive experimental evaluation validates the CF-DETR’s effectiveness when orchestrated by an NPFP** policy. Conducted on representative server and GPU-enabled embedded platforms, and demonstrated on an actual AV platform, CF-DETR consistently achieved superior detection accuracy (overall and critical mAP) compared to baselines like standard DETR scheduling and state-of-the-art real-time DNN (RT-DNN) systems (e.g., DNN-SAM [7]). This enhanced accuracy, particularly for vital critical detections, was attained

with competitive or improved throughput (FPS), effectively managing the latency-accuracy trade-off while ensuring robust performance and precise localization crucial for AV safety.

Contribution. Our approach fundamentally diverges from existing methods through its Transformer-specific co-design, without requiring additional sensors like LiDAR. Unlike generic RT-DNN systems (e.g., DNN-SAM), unaware of internal model characteristics (like attention mechanisms or patch granularity sensitivity) and thus allocating resources inefficiently for DETR models, our method leverages deep insights into these properties for effective optimization (A1 and A2). Furthermore, it incorporates Transformer-aware batching techniques (such as patch-level batch inference, A3) that yield significantly higher accuracy gains relative to computational cost compared to conventional batch-DNN methods ill-suited for complex Transformer workloads. This holistic, model-aware strategy addresses the limitations of applying generic real-time or batching solutions to multi-DETR systems’ unique challenges. Our work is distinct from CF-ViT [13] in two critical aspects: architecturally, CF-DETR leverages decoder-based object queries to refine uncertain regions for detection, opposing CF-ViT’s encoder-based refinement of informative regions for classification. Systemically, CF-DETR is integrated within a real-time scheduling framework to manage latency-accuracy trade-offs, entirely outside the scope of CF-ViT.

The key contributions of this paper are:

- We propose the CF-DETR incorporating novel strategies A1–A3 exploiting Transformer properties to efficiently mitigate DETR’s latency-accuracy trade-off.
- We introduce the NPFP** scheduling framework A4, encompassing various instantiable policies, synergistically integrating A1–A3 strategies to achieve R1 and R2.
- We provide extensive empirical validation, including evaluations on an actual AV platform, demonstrating that the CF-DETR significantly outperforms baselines in both overall detection accuracy (mAP) and real-time performance (FPS) across diverse AV workloads, ensuring critical object responsiveness and precise localization.

II. TARGET SYSTEM AND OBSERVATION

This section explains our target system and presents key observations from a measurement-based case study that inform the design of CF-DETR.

A. Target system: detection Transformer

AVs typically rely on multiple cameras (e.g., front, side, rear) that each run a multi-object detection (MOD) model to spot vehicles, pedestrians, and other obstacles in every frame. Unlike traditional convolutional detectors (e.g., YOLO series [14]) that only capture local features, DETR employs a patch-based representation and a Transformer architecture to model global context. The input image is first divided into a grid of small patches (e.g., a 3×3 grid in Fig. 1(a)), and each patch is encoded as an embedding token (i.e., feature extraction, converting from RGB to a computational vector). The Transformer encoder then applies self-attention over all

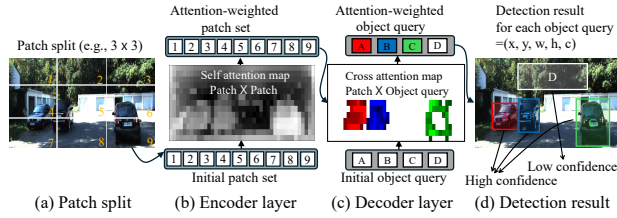


Fig. 1. Overview of the DETR pipeline for object detection. The input image is divided into patches (a), which are encoded and processed via self-attention in the Transformer encoder (b). Learned object queries in the Transformer decoder interact with encoded patches through cross-attention (c), producing bounding boxes, class labels, and confidence scores (d).

patch tokens, meaning each patch attends to every other patch to capture their relationships. This produces an attention-weighted set of patch features enriched with global context (visualized as a patch-to-patch attention map in Fig. 1(b)). As a result, even distant parts of the image can influence each other's features, helping the model recognize large-scale structures as well as contextual cues for object detection.

After encoding, DETR's Transformer decoder produces the object detections by interacting with a fixed set of learned object queries. These object queries can be thought of as placeholders for potential objects in the image. In the decoder, each query attends to the encoded patch features (through encoder-decoder cross-attention) and gradually updates its representation to focus on a specific object. In Fig. 1, the object queries (A, B, C, and D in Fig. 1(c)) attend to different patches (illustrated by patch-query attention maps), and each query outputs a bounding box with a class label (e.g., car or pedestrian) and a confidence score. High-confidence detections (boxes of A, B, C in Fig. 1(d)) typically correspond to prominent objects, while low-confidence outputs (a box of D in Fig. 1(d)) may indicate a background or small objects that the model is less certain about.

B. Observation

To design a DETR that satisfies both timing guarantee (R1) and high accuracy (R2), we first examine how patch granularity affects DETR's performance. We conducted a case study using the DINO [15], a state-of-the-art DETR model, on the KITTI dataset [16], evaluating performance on an NVIDIA Jetson Orin system-on-chip (SoC) [17]. From this study, two key empirical observations O1 and O2 emerge.

O1. Latency-accuracy trade-off: There is a clear trade-off between detection accuracy and inference speed depending on the patch size.

Fig. 2(a) illustrates that using coarser patches (e.g., the number of 795) yields faster inference (e.g., 129 ms) but at the cost of lower detection accuracy (averaged across all input images). Using finer patches (e.g., the number of 11394) improves accuracy, but incurs much higher latency (e.g., 226 ms). This indicates that an extremely coarse division (very few patches) might run quickly but only detect large, obvious objects (compromising R2), whereas a very fine division

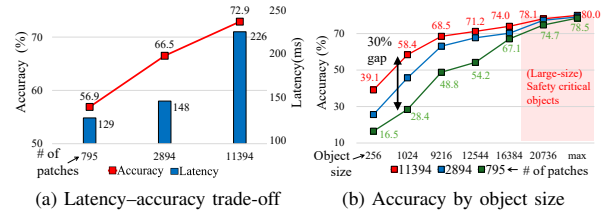


Fig. 2. Accuracy and latency analysis of DETR (DINO model) with varying patch granularity on the KITTI dataset (evaluated on Jetson Orin). (a) Trade-off between accuracy (averaged across all images) and latency with different numbers of patches. (b) Accuracy (averaged for each object-size group) comparison by object size, showing coarse patches are sufficient for large objects but significantly reduce accuracy for smaller objects.

greatly boosts accuracy on small objects but could violate the real-time latency requirements (compromising R1). This fundamental trade-off implies that no single patch resolution can simultaneously satisfy both R1 and R2.

O2. Object size vs. patch granularity: Large, safety-critical objects are reliably detected with coarse patches, while small objects require finer granularity for high accuracy.

Fig. 2(b) supports this insight by breaking down accuracy according to object size, with the x-axis representing object area (width×height pixels). DETR using coarse patches achieves accuracy (averaged per object-size group) nearly comparable to that with fine patches for large objects, such as nearby vehicles occupying an area exceeding 16384 pixels. However, accuracy significantly deteriorates for smaller objects, showing roughly a 30% accuracy gap between coarse (green curve) and fine settings (red curve). This confirms that coarse patches are sufficient for large objects, but finer patches are required to reliably detect smaller ones.

These observations highlight a fundamental challenge O1 in employing DETR for AV perception, while also providing a critical insight O2 toward overcoming it. Specifically, high accuracy is achievable with coarse patches (at low latency) for images containing only large objects. For images featuring varying object sizes, selectively applying coarse patches to large-object regions and fine patches to small-object regions maintains high accuracy while incurring lower latency compared to uniformly using fine patches.

CF-DETR systematically exploits this DETR-specific property and employs specialized batching techniques A1–A3 to navigate the accuracy-latency trade-off. It also integrates a scheduling framework A4 ensuring timing guarantees. These system-level design elements are detailed in the next section.

III. SYSTEM DESIGN

This section details the design of CF-DETR, encompassing its core architectural strategies and the real-time scheduling framework, followed by experimental validation of its key components' functionality.

A. Overview

To satisfy both real-time and accuracy requirements R1 and R2, CF-DETR leverages insights from our experimental

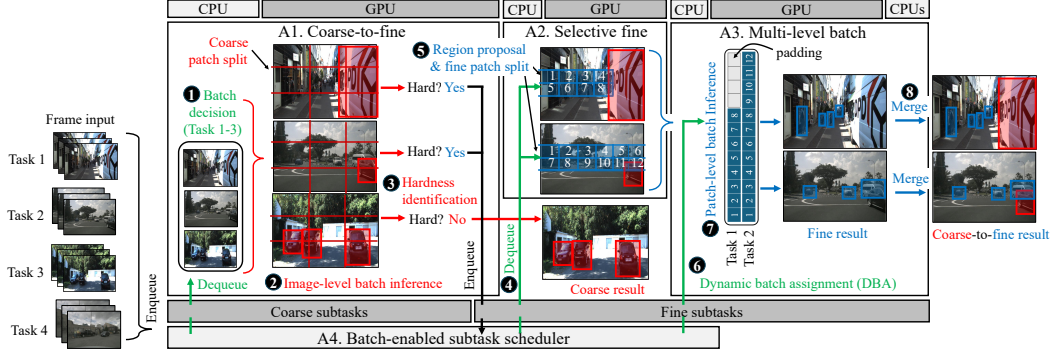


Fig. 3. System overview of CF-DETR: Periodic frames from multiple tasks arrive and queue. The scheduler then forms initial processing batches (❶). Inputs undergo coarse batch inference using a low-res grid—yielding initial detected boxes through concurrent DETR inferences in a single batch (❷). Frames are classified by confidence as *easy* (high-confidence) or *hard* (ambiguous) (❸); this determines if fine subtasks are needed. Easy frames use coarse results directly. Hard frames generate fine subtasks, which are queued for refinement. The scheduler determines their execution timing (❹). For each fine subtask, hard regions are identified and subdivided into finer patches, defining its workload (❺). The scheduler selects a dynamic fine batch configuration from active subtasks, maximizing throughput within deadlines (❻). Fine subtasks use the patch-level batch inference to refine detections in uncertain regions (❼). Finally, refined outputs merge with high-confidence coarse detections for the final results (❽).

observations in Sec II-B and employs four key synergistic strategies: (A1) coarse-to-fine inference, (A2) selective fine inference, (A3) multi-level batch inference, and (A4) a batch-enabled subtask scheduler. The overall system architecture and workflow, which demonstrates how these strategies process incoming frames from multiple tasks—initiating with a coarse subtask, followed by adaptive fine-grained processing and batching, all orchestrated by the real-time scheduler—are depicted in Fig. 3. The detailed design of these components is elaborated in Sec. III-B.

Benefits. Leveraging its coarse-to-fine paradigm and integrated scheduling, CF-DETR offers several key benefits. First, its broad applicability allows compatibility with most (if not all) Transformer-based object detectors by utilizing common architectural components. Second, it ensures prioritized responsiveness and accuracy for large, safety-critical objects, which are typically detected rapidly and reliably in the coarse stage (A1), crucial for safety in autonomous systems. Third, CF-DETR maximizes overall detection accuracy while adhering to deadlines by adaptively scaling fine-stage computation (A2) based on available slack time. Finally, it enhances GPU throughput via multi-level batch inference (A3), a specialized batch processing scheme for Transformer computations, effectively parallelizing inference across multiple tasks.

B. Design of CF-DETR

We now detail the design and empirical validation of the key components of CF-DETR. The system dedicates a thread for each camera and another for the scheduler, with inter-thread communication facilitated by shared memory. Scheduler operations are performed on the CPU, while DNN inference is executed on the GPU.

A1. Coarse-to-fine inference (frame-level: ❷–❸). Incoming frames periodically generate coarse subtasks, which are enqueued. The scheduler determines their execution timing and whether to process them individually or in a image-level

batch. These subtasks perform rapid detection using coarse-grained patches (with a predefined granularity) (❷). This initial inference yields, for each object query, a candidate object’s location (x, y) , size (w, h) , and confidence score (c) , where $0 \leq c \leq 1$. Based on these results, frames are classified by their difficulty (❸). Specifically, after excluding objects with high confidence (e.g., $c > 0.8$), typically deemed large and safety-critical, an average confidence score is computed for the remaining candidates within each frame. If this average is below a certain threshold (e.g., 0.05), the frame is classified as “easy”; its coarse results are considered sufficient, and fine-grained processing is skipped. Such a low average implies that, after high-confidence critical objects are accounted for, the remaining detections predominantly correspond to background regions. Conversely, frames with higher average confidence scores (indicating more ambiguity or complexity, often due to smaller or less distinct non-critical objects) are labeled “hard,” triggering selective refinement.

Validating ❸. Fig. 4(a) visualizes a sample coarse inference output (DINO model, 795 patches, KITTI), with color-coded boxes indicating ground truth (yellow), high-confidence queries ($c > 0.8$, red) for prominent objects, intermediate-confidence queries ($0.05 < c \leq 0.8$, blue) flagged for refinement, and background-indicative queries ($c \leq 0.05$, green). To determine frame difficulty, we analyzed object query confidence distributions across KITTI after coarse inference and filtering of high-confidence queries, defining an “easy set” (frames with ≤ 5 remaining intermediate-confidence queries) and a “hard set” (frames with > 10). As shown in Fig. 4(b), the average query confidence distributions distinctly separate easy set (highly skewed, near-zero confidences) from hard set (flatter distributions with more intermediate confidences); this distinction enables CF-DETR to trigger fine inference selectively and underpins our use of an average confidence threshold (e.g., 0.05) to classify frames as easy or hard.

A2. Selective fine inference (region-level: ❺). For frames

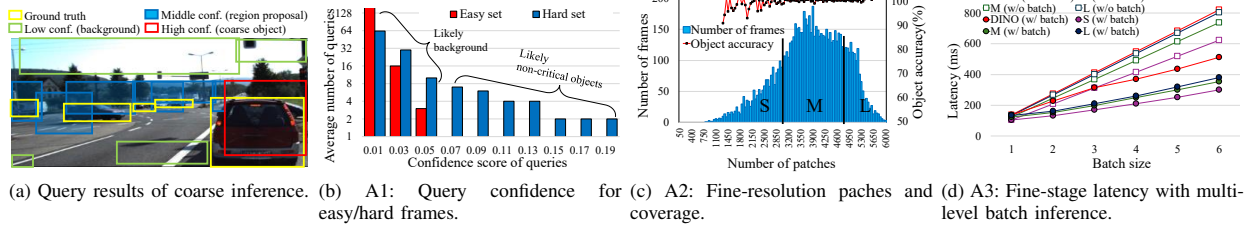


Fig. 4. Validation of CF-DETR’s adaptive inference strategies A1–A3 using the DINO model on Jetson Orin with the KITTI dataset: (a) sample coarse inference output; (b) hardness identification by query confidence analysis; (c) selective fine inference efficiency, showing patch count versus object coverage; and (d) multi-level batch inference latency advantages for fine-stage processing.

classified as “hard” by A1, CF-DETR performs region proposal and fine patch split (5), concentrating computation on challenging regions of interest (ROIs) instead of the entire image. These ROIs are proposed based on the locations (x, y) and sizes (w, h) of intermediate-confidence queries from the A1 coarse inference stage. Critically, only these query-defined regions are subsequently re-partitioned into finer patches, which are then used to process the selected ROIs, thereby enhancing local resolution. Meanwhile, non-ROI areas reuse the original coarse-stage tokens from A1.

Validating 5. Fig. 4(c) illustrates the distribution of fine-resolution patch counts generated by CF-DETR’s region proposal and selective patch split mechanism (5). This performance is benchmarked against a DINO model that typically requires more than 10,000 patches for equivalent full-frame fine resolution. The figure also plots corresponding object accuracy (%), indicating the ground-truth coverage within these selectively refined regions. Consequently, CF-DETR processes the hardest frames with under 6,000 fine-resolution patches—a mere fraction of DINO’s more than 10,000 for equivalent full resolution—achieving a significant latency advantage yet maintaining comparable accuracy by capturing nearly 100% of ground-truth objects in these selectively refined areas.

A3. Multi-level batch inference (batch-level: 7–8). To enable latency-effective batched execution of fine-grained patches derived from A2-identified hard frames, CF-DETR introduces *patch-level* batch inference. This approach avoids the accuracy degradation common in traditional *image-level* batching (arising from input cropping/resizing to meet identical input size constraints, which can cause information loss) by leveraging A2’s already feature-extracted fine patches. These patches originate from A2’s validated region proposals (6), which themselves exhibit high ground-truth coverage (per Fig. 4(c)). Since patch counts from A2 can vary across tasks, padding is applied to equalize them for batch configuration and assignment (7) before they are processed in the subsequent patch-level batch inference step (8). This patch-level method offers two key advantages over image-level batching: 1) it eliminates redundant backbone feature re-extraction (cf. Fig. 1(a)), and 2) it achieves substantial latency improvements by processing only these targeted, high-coverage patches. Note that padding is only strictly necessary for the inter-token attention layers within Transformer architectures, as linear layers (e.g., QKV projection, FFNs) could process variable-sized inputs batched at the patch level. However, avoiding

padding for these intra-token operations necessitates a complex dynamic token packing/unpacking mechanism within every Transformer block. This approach incurs substantial runtime overhead from intricate index management, frequent memory copy operations, and continuous tensor reshaping.

Validating 7. Patch-level batch inference yields substantial latency benefits, as shown in Fig. 4(d). This figure illustrates that compared to processing fine tasks individually (empty squares), batched execution (circles) significantly reduces total latency; for instance, six concurrent fine inferences achieve an approximate $2\times$ speedup. Furthermore, Fig. 4(d) compares the inference times of conventional image-level batching for DINO (red circles) against CF-DETR’s patch-level batch inference (after region proposal 5). For this comparison, patches derived from hard frames were evaluated, based on analysis related to Fig. 4(c), where patch counts like 3000 and 4800 were used as thresholds to categorize into small (S), medium (M), and large (L) sets for simplicity. The S, M, and L sets (marked by purple, green, and blue circles, in Fig. 4(d)) show significant improvements: at a batch size of six, DINO’s image-level approach took approximately 500 ms, whereas CF-DETR’s patch-level processing required only around 300 ms for S-set patches and 400 ms for L-set patches, demonstrating latency reductions of approximately 20–40%.

A4. Batch-enabled subtask scheduler (1, 4, and 6). The scheduler is batch-aware: it forms initial coarse processing batches (1) and also handles the dynamic assignment of fine tasks into batches (6) for execution via A3 when feasible. Operationally, incoming frames are dispatched for coarse processing. Hard frames identified by A1 generate fine subtasks, which are queued. When the GPU is free from high-priority coarse work, the scheduler determines their execution timing (4) and executes these pending fine subtasks, often as optimally configured batches (utilizing 6) from the queue. This coordination ensures critical deadlines are met while maximizing timely fine-grained refinements. Sec. IV details this scheduler’s operation.

Model training. CF-DETR uses a single, unified model for coarse and fine inference, addressing memory effectiveness concerns. The Transformer architecture handles dynamic token counts without altering parameter size. To ensure shared the same weights, CF-DETR employs a unique training strategy with a new loss function: after COCO pre-training, each iteration performs coarse and fine passes, calculating respective losses and updating parameters with a combined loss,

while selectively freezing the backbone. This unified, memory-efficient approach yields a negligible accuracy difference (< 1% mAP) compared to separate models.

IV. SCHEDULING FRAMEWORK

This section presents the task model and the NPFP** scheduling framework developed for CF-DETR.

A. Task model

We consider a multi-object detection system comprising n strictly-periodic DETR tasks, $\tau = \{\tau_1, \dots, \tau_n\}$. These tasks execute on a heterogeneous platform (multiple CPUs, single shared GPU), such that their CPU and GPU processing components execute exclusively. Where advantageous, parallelizable CPU operations can utilize multi-core threading, while accounting for inter-core communication costs (e.g., ③ in Fig. 3). Each task $\tau_i \in \tau$ is defined by its period T_i and a relative deadline D_i ; we assume $D_i = T_i$ for all tasks.

A central concept of the CF-DETR model is the partitioning of each task τ_i into two sequential components: a safety-critical coarse subtask τ_i^S , followed by an optional fine subtask τ_i^F . Formally, the task structure is represented as $\tau_i = (\tau_i^S, \tau_i^F, T_i, D_i)$. The subtasks are parameterized as follows:

- The coarse subtask $\tau_i^S = (p^S, C_i^S)$ processes a fixed, pre-determined number of coarse patches, p^S , with the profiled execution time (PET) of C_i^S . The decision to subsequently invoke the fine subtask τ_i^F is made during the execution of τ_i^S , specifically within its hardness determination process (cf. ③ in Fig. 3).
- The fine subtask $\tau_i^F = (p_i^F, C_i^F(p_i^F))$ processes a dynamically determined number of fine patches, p_i^F , which can be one of several predefined levels (e.g., S, M, or L, corresponding to $0 < S < M < L$ patches), or zero if the fine subtask is skipped. The value of p_i^F is identified during the selective patch splitting stage (cf. ⑤ in Fig. 3) performed at the beginning of τ_i^F itself. Consequently, its PET, $C_i^F(p_i^F)$, is contingent upon this chosen p_i^F . If no fine processing is required, τ_i^F is skipped (effectively $p_i^F = 0$, and $C_i^F(0) = 0$).

The PETs, C_i^S and $C_i^F(p_i^F)$, are calculated based on their constituent operations:

$$C_i^S = c_i^{ps} + c_i^{at}(p^S) + c_i^{dt}, \quad (1)$$

$$C_i^F(p_i^F) = c_i^{sps}(p_i^F) + c_i^{at}(p_i^F), \quad (2)$$

where c_i^{ps} is the PET for coarse patch splitting, $c_i^{at}(p)$ is for the attention mechanism over p patches ($p = p^S$ for τ_i^S , $p = p_i^F$ for τ_i^F), c_i^{dt} is for hardness determination, and $c_i^{sps}(p_i^F)$ is for selective patch splitting (which itself depends on the outcome that determines p_i^F). Task periods T_i may differ (e.g., for different camera types), though the model also applies to uniform periods. A job is considered active at time t if it has been released but not yet completed. We use $\tau^S(t)$ and $\tau^F(t)$ to denote the sets of active coarse and fine subtasks at time t , respectively, and $r_i(t)$ to denote the next release time

Algorithm 1 The NPFP** generic algorithm

At scheduling instant t (job completion or arrival on idle system):

```

1: if  $\tau^S(t) \neq \emptyset$  then
2:   if The first "*" in NPFP** represents C then
3:     Execute the highest-priority active coarse task in  $\tau^S(t)$ .
4:   else if The first "*" in NPFP** represents [C] then
5:     Execute a coarse batch using Algo. 2.
6:   end if
7: else if  $\tau^F(t) \neq \emptyset$  then
8:   if The second "*" in NPFP** represents F then
9:     Execute the highest-priority active fine task in  $\tau^F(t)$  if its
       PET plus current time  $t$  is no later than the earliest future
       release of any coarse subtask.
10:  else if The second "*" in NPFP** represents [F] then
11:    Execute fine batch(s) using Algo. 3.
12:  end if
13: end if

```

of task τ_i after or at t . For periodic tasks, release times and deadlines are deterministic. The coarse subtask τ_i^S must be completed by its deadline D_i ; if its corresponding fine subtask τ_i^F is subsequently initiated, it executes after τ_i^S and must also conclude by this same deadline D_i .

The PET values for individual coarse subtasks (C_i^S) and fine subtasks ($C_i^F(p_i^F)$), along with those for their batched executions ($C_{\mathcal{B}^S}$ and $C_{\mathcal{B}^F}$, where $\mathcal{B}^S$ and $\mathcal{B}^F$ are sets of batched coarse and fine subtasks, respectively, with $|\mathcal{B}^S| \geq 2$ and $|\mathcal{B}^F| \geq 2$), are determined through offline, measurement-based PET analysis, as detailed in Sec. V. As a property of batch execution, we assume $C_{\mathcal{B}^S} \leq \sum_{\tau_i \in \mathcal{B}^S} C_i^S$ (and similarly for $C_{\mathcal{B}^F}$). CF-DETR exclusively relies on these pre-determined offline PETs, not on runtime predictions. Achieving predictable PETs is feasible given uniform input sizes and dedicated GPU utilization per DNN type, which avoids resource contention [6], [7], [11]. This methodology aligns with recent advancements in execution time analysis for DNNs [18]. We operate under the assumption that a reliable upper bound on execution time can be established through intensive measurements complemented by appropriate safety margins.

B. The NPFP** framework

The NPFP** framework establishes a unified non-preemptive fixed-priority (NPFP) scheduling approach for the CF-DETR system. To delineate the various execution strategies within this framework, we adopt the following notation for the superscript of NPFP: C and F denote the individual (non-batched) execution of coarse (τ_i^S) and fine (τ_i^F) subtasks, respectively. When enclosed in brackets, [C] and [F] signify the batched execution of coarse and fine subtasks, respectively. This framework accommodates these different modes while aiming to ensure timing guarantees. Given that fine subtask execution (F or [F]) is optional and opportunistic, several instantiations of the NPFP** framework can be realized, such as NPFP^C (C-only), NPFP^{C[F]}, NPFP^{[C]F}, and NPFP^{[C][F]}. While individual subtasks (either C or F) run non-preemptively once initiated, preemption by a higher-priority

task is permitted, but only at the distinct boundary between the completion of a task's coarse subtask and the potential initiation of its fine subtask.

Principle. A fundamental rule of NPFP** framework is that coarse subtasks (τ_i^S) always possess higher scheduling priority than fine subtasks (τ_i^F). This ensures that safety-critical processing is prioritized; consequently, fine subtasks can only commence execution when no coarse subtasks are ready. Among coarse subtasks (likewise fine subtasks), a pre-defined priority is assigned. The offline schedulability guarantee primarily covers the individual execution of all coarse subtasks (C). Decisions regarding batched coarse execution ([C]), or any fine subtask execution (either individual, F, or batched, [F]), are made dynamically at runtime. These runtime choices aim to maximize accuracy by utilizing available resources, always contingent upon not compromising the pre-established schedulability of the critical coarse subtasks; critically, the overhead of any runtime schedulability analysis required for these decisions must be *minimal*.

Algo. 1 illustrates the generic decision logic within the NPFP** framework at each scheduling instant t . It strictly prioritizes active coarse subtasks ($\tau^S(t)$) over active fine subtasks ($\tau^F(t)$). If any coarse subtasks are active (Line 1), a decision is made according to the specific NPFP policy variant (denoted by “***”). Depending on the policy, the system either selects the highest-priority individual coarse subtask (Line 3, if the policy includes C) or invokes Algo. 2 to schedule a batch of coarse subtasks (Line 5, if the policy includes [C]). Note that Algo. 2 may still choose to execute only the highest-priority active coarse subtask, depending on its run-time schedulability test result. If no coarse subtasks are active but fine subtasks are (Line 7), a similar policy-dependent decision is applied: either executing the highest-priority individual fine subtask (Line 9, if the policy includes F, contingent on its PET being less than or equal to the time until the earliest future coarse subtask release) or a batch of fine subtasks (Line 11, if the policy includes [F], employing a dynamic fine batch assignment strategy in Algo. 3 to be executed). The specific conditions guiding the choice between individual and batched execution (represented by “***”) and the referenced batch formation algorithms are defined by the particular NPFP** instantiation (e.g., NPFP^{[C]F} or NPFP^{[C][F]}). The foundational analysis of NPFP^C, the simplest variant involving only individual coarse subtask execution, establishes the baseline for system schedulability guarantees.

Following NPFP rules, scheduling decisions occur at discrete instants t , triggered by job completions (if other jobs are pending) or new job arrivals on an idle platform. At each such point t , the highest-priority ready coarse subtask τ_i^S from the set of active coarse subtasks $\tau^S(t)$ is selected and runs non-preemptively for its duration C_i^S . The offline schedulability for NPFP^C is given by Lemma 1.

Lemma 1 (NPFP schedulability test [19]–[21]): A task set τ scheduled by NPFP^C is schedulable, if every $\tau_i \in \tau$ satisfies Eq. (3).

$$R_i \leq T_i, \quad (3)$$

where the upper-bounded response time R_i is the smallest positive value found by iterating Eq. (4) until convergence, i.e., $R_i(x+1) = R_i(x)$, starting with $R_i(0) = C_i^S + B_i$:

$$R_i(x+1) = C_i^S + B_i + \sum_{\tau_h \in \text{HP}(\tau_i)} \left\lceil \frac{R_i(x)}{T_h} \right\rceil \cdot C_h^S. \quad (4)$$

Here, $B_i = \max(\{0\} \cup \{C_j^S \mid \tau_j \in \text{LP}(\tau_i)\})$ denotes the maximum blocking from lower-priority tasks of τ_i ($\text{LP}(\tau_i)$), and $\text{HP}(\tau_i)$ represents the set of higher-priority tasks of τ_i .

Proof: We analyze a level- i busy period. Without loss of generality, let this period start at $t = 0$ with job J_i of task τ_i . The critical instant theorem [22] (Lemma 6) states τ_i 's upper-bounded response time occurs with synchronous higher-priority ($\text{HP}(\tau_i)$) arrivals at $t = 0$, after maximum blocking $B_i = \max(\{0\} \cup \{C_j^S \mid \tau_j \in \text{LP}(\tau_i)\})$ from a lower-priority task in $\text{LP}(\tau_i)$. In an interval $[0, L]$, the higher-priority task workload is at most $\sum_{\tau_h \in \text{HP}(\tau_i)} \lceil \frac{L}{T_h} \rceil \cdot C_h^S$. Eq. (4)'s fixed point, R_i , sums C_i^S , B_i , and higher-priority interference, bounding the time to finish J_i . Thus, if $R_i \leq T_i$ (Eq. (3)), J_i meets its deadline. This establishes the base case.

For the inductive step, we show job separation $t_{x+1} - t_x \leq T_i$ within the busy period. At t_x (the start time of the x^{th} job), no higher-priority jobs are pending. The time for C_i^S plus higher-priority interference within $[t_x, t_x + L]$ is $R'_i(L) = C_i^S + \sum_{\tau_h \in \text{HP}(\tau_i)} \lceil \frac{L}{T_h} \rceil \cdot C_h^S$, which omits B_i . Since $R'_i(L)$ is bounded by the value from Eq. (4) (including B_i), the premise $R_i \leq T_i$ ensures the solution L^* to $L = R'_i(L)$ also satisfies $L^* \leq T_i$. This means the x^{th} job's load resolves within T_i , allowing the $(x+1)^{\text{th}}$ job to start by $t_x + T_i$. Hence $t_{x+1} - t_x \leq T_i$, consistent with anti-self-pushing results [20]. The base and inductive steps prove the lemma by induction. ■

C. Coarse batch assignment

The coarse batch assignment mechanism is invoked when an NPFP** policy supporting batched coarse execution ([C]) is active (e.g., as called by Line 5 of Algo. 1, referring to Algo. 2 detailed herein). Coarse batching may impact the schedulability guaranteed by Lemma 1 for individual coarse subtasks. As runtime verification of schedulability can incur significant overhead, a core objective for any NPFP** variant implementing [C] is to minimize this runtime analysis cost. Our approach achieves this by adopting a carefully designed batch policy with systematic constraints. While drawing inspiration from methods like Batch-MOT [18], we incorporate additional constraints that dramatically simplify the runtime analysis by ensuring batch execution aims for theoretical simplicity and low runtime cost. As demonstrated in Sec. V, this constrained method achieves near-maximal accuracy, suggesting complex runtime batching analyses may primarily add overhead without proportional benefits.

Principle. To minimize the runtime schedulability analysis cost, our design for coarse batching is founded on two key principles.

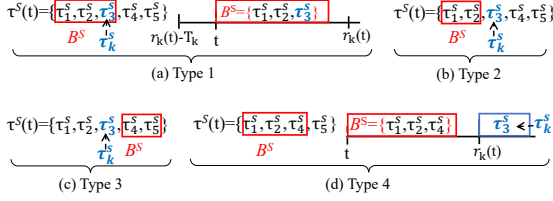


Fig. 5. Four types of coarse subtask batching scenarios ($\mathcal{B}^S$) illustrating potential impacts on task executions under the NPFP** framework

- P1. Only batches consisting of a contiguous sequence of active coarse subtasks in $\tau^S(t)$, ordered from highest to lowest priority and always including the highest-priority coarse task, are considered permissible.
- P2. Any coarse batch execution starting at time t must complete before the earliest future release time of any task in the system, i.e.,

$$C_{\mathcal{B}^S} \leq \min_{\tau_i \in \tau^S} r_i(t) - t. \quad (5)$$

where $r_i(t)$ is the earliest release time of τ_i after or at t .

Consider a task set τ deemed schedulable under NPFP^C (per Lemma 1). If a [C]-enabled scheduler executes a coarse batch $\mathcal{B}^S$ at time t , its potential impact on any job J_k (of $\tau_k^S \in \tau$) is categorized into four scenario types (illustrated in Fig. 5). For this analysis, let τ_h^S be the highest-priority task in $\mathcal{B}^S$, $\tau^S(t)$ be the set of active coarse subtasks at t , and $r_k(t)$ be the earliest release of τ_k after t . P1 and P2 (in conjunction with the batching property) guarantee that run-time coarse batching for each type preserves the schedulability ensured by Lemma 1, as follows. Recall that the batching property represents $C_{\mathcal{B}^S} \leq \sum_{\tau_i \in \mathcal{B}^S} C_i^S$.

- Type 1: $\tau_k^S \in \tau^S(t)$ and $\tau_k^S \in \mathcal{B}^S$ (Fig. 5(a))
- Type 2: $\tau_k^S \in \tau^S(t)$, $\tau_k^S \notin \mathcal{B}^S$, and $\tau_h^S \in \text{HP}(\tau_k^S)$ (Fig. 5(b))

Under scenarios of Type 1 or 2, the higher-priority interference to tasks not in $\mathcal{B}^S$ will decrease (by the batching property), and the tasks in $\mathcal{B}^S$ finish their batching before their deadlines (by P2).

- Type 3: $\tau_k^S \in \tau^S(t)$, $\tau_k^S \notin \mathcal{B}^S$, and $\tau_h^S \in \text{LP}(\tau_k^S)$ (Fig. 5(c))
Type 3 scenarios are ruled out by P1.
- Type 4: $\tau_k^S \notin \tau^S(t)$ (Fig. 5(d))

Under Type 4 scenarios, τ_k is not affected by the batching execution (due to P2). The impact of the batching on tasks in $\mathcal{B}^S$ and those not in $\mathcal{B}^S$ is equivalent to that observed in scenarios of Type 1 or 2.

Algo. 2 details the runtime logic for coarse batch assignment within any NPFP** instantiation that supports [C]. At each scheduling instant t , Algorithm 2 first checks if multiple coarse subtasks are active ($|\tau^S(t)| \geq 2$) (Line 1). If so, candidate batches $\mathcal{B}^S(k)$ (the k highest-priority active coarse tasks) are defined (Line 2). The algorithm then seeks the largest $x \geq 2$ such that batch $\mathcal{B}^S(x)$ satisfies the runtime condition Eq. (5) (Line 3). If such an x is found (Line 4), this batch

Algorithm 2 Coarse batch execution

```

1: if  $|\tau^S(t)| \geq 2$  then
2:   Let  $\mathcal{B}^S(n)$  be the set of the  $n$  highest-priority tasks in  $\tau^S(t)$ ,  $1 \leq n \leq |\tau^S(t)|$ .
3:   Find the largest  $x$  ( $2 \leq x \leq |\tau^S(t)|$ ) s.t.  $\mathcal{B}^S(x)$  satisfies Eq. (5).
4:   if such an  $x$  exists then
5:     Execute tasks in  $\mathcal{B}^S(x)$  as a batch, and return.
6:   end if
7: end if
8: Execute the single highest-priority coarse active job, and return.

```

$\mathcal{B}^S(x)$ is executed (Line 5). Otherwise (if $|\tau^S(t)| < 2$ or no such x is found), Line 8 executes the default: the single highest-priority active coarse subtask is run. This mechanism ensures that coarse batching is only employed when deemed safe by the runtime check. If no coarse subtasks are eligible for execution (either batched or individual), the overarching NPFP** scheduler (Algo. 1) would then consider fine subtask execution based on its policy (e.g., Lines 7–12 of Algo. 1).

Lemma 2: If a task set τ is deemed schedulable under NPFP^C (per Lemma 1), τ is also schedulable under NPFP^[C].

Proof: (Active τ_k) By P1, there is no scenario in which the batching lower-priority task(s) without including τ_k itself. Thus, two cases arise: (i) the batch does not include any lower-priority tasks, or (ii) the batch includes one or more lower-priority tasks along with τ_k . In case (i), due to the batching property, such batching does not increase the PET of higher-priority interference, or of τ_k itself if it is included. In case (ii), the schedulability of τ_k is guaranteed by P2. In both cases, P2 ensures that the batching execution does not affect any future job releases of τ_k .

(Inactive τ_k) As explained in Type 4 scenarios, P2 guarantees that batching execution does not interfere with future job releases of τ_k . ■

D. Fine batch assignment

The fine batch assignment mechanism, detailed in Algo. 3, is invoked by an NPFP** policy that supports batched fine subtask execution ([F]). This typically occurs when Line 11 of the generic Algo. 1 is reached, specifically at a time t when no coarse subtasks are active ($\tau^S(t) = \emptyset$) but active fine subtasks ($\tau^F(t) \neq \emptyset$) exist. Upon invocation, the system first determines the workload (i.e., patch count $p^F \in \{S, M, L\}$ based on categories like small (S), medium (M), large (L) from patch splitting, cf. Fig. 3 ⑤) for all $\tau_i^F \in \tau^F(t)$. These fine subtasks are then designated for execution using patch-level batching, potentially leveraging mechanisms like multi-level batch inference (cf. Fig. 3 ⑦). A primary challenge in such fine subtask batching is the padding overhead: inputs with heterogeneous patch counts within a single batch are typically padded to match the maximum patch count (cf. Fig. 3 ⑥), incurring latency costs dependent on batch composition. Furthermore, two critical timing constraints must be satisfied: (i) each fine subtask must meet its own deadline for its individual schedulability, and (ii) the entire sequence of fine subtask batches must complete execution before the earliest future

Algorithm 3 Fine batch execution

- 1: **if** $|\tau^F(t)| \geq 2$ **then**
 - 2: Let $\tau^F(t)'$ be the sequence of active fine subtasks from $\tau^F(t)$, sorted non-decreasingly by workload (e.g., patch counts).
 - 3: Determine an partition of $\tau^F(t)'$ into contiguous batches $B_1^F, \dots, B_x^F$ that minimizes total predicted PET $\sum_{y=1}^x C_{B_y^F}$ (per Lemma 4).
 - 4: Execute the batch sequence $B_1^F, \dots, B_x^F$, respecting fine subtask deadlines and the earliest future coarse subtask release, then *return*.
 - 5: **end if**
 - 6: Execute the single highest-priority active fine subtask, if its PET plus current time t is no later than the earliest future release of any coarse subtask, then *return*
-

release time of any coarse subtask to ensure coarse subtask schedulability is not compromised.

Finding a partition of the $|\tau^F(t)|$ active fine subtasks (with their determined workloads w_i) into batches $B_1^F, \dots, B_x^F$ that minimizes the total predicted PET $\sum_{y=1}^x C_{B_y^F}$ while satisfying constraints (i) and (ii) is computationally exponential (potentially $O(2^{|\tau^F(t)|})$). Each batch PET, $C_{B_y^F}$, is significantly influenced by padding overheads from mechanisms like multi-level batch inference; for instance, if subtasks with different workloads (e.g., S, M, L) are batched, all are processed based on the maximum workload (e.g., L), and $C_{B_y^F}$ reflects the PET for this padded configuration. Our goal is therefore to develop an efficient heuristic that reasonably reduces padding overhead and identifies a schedulable set of batches. To efficiently partition active fine subtasks while reasonably reducing padding overhead, we propose the dynamic batch assignment (DBA) algorithm.

Principle. The core principle of DBA is to group fine subtasks with similar workloads, aiming to reduce PET penalties from padding. As detailed in Algo. 3, DBA first employs a heuristic step of sorting the $|\tau^F(t)|$ active fine subtasks by their workloads (e.g., patch counts) into a non-decreasing sequence $\tau^F(t)' = (w_1, \dots, w_{|\tau^F(t)|})$ (Line 2). Restricting batch formation to only *contiguous* subtasks within this sorted sequence significantly reduces the search space for partitioning. Within this setting, DBA applies dynamic programming (DP) to determine a partition of $\tau^F(t)'$ into contiguous batches (Line 3). This DP computation, guided by the recurrence in Eq. (6) and detailed in Lemma 4, inherently incorporates system constraints. Specifically, if potential batches or sub-partitions violate either (i) individual fine subtask deadlines or (ii) the completion deadline set by the earliest next coarse subtask release, they are assigned an effective infinite cost and are thus excluded by the min operation in the recurrence.

The DP stage, therefore, constructs a schedulable partition (if one exists satisfying the integrated constraints) for $\tau^F(t)'$ with an $O(|\tau^F(t)|^2)$ time complexity. The resulting DP-derived partition is then processed by Line 4 of Algo. 3 to dispatch an executable batch sequence. First, we establish a foundational result for minimizing total execution time without considering deadlines. This forms the core optimization

TABLE I
DP COMPUTATION EXAMPLE (DBA[4]=5)

Step k	1	2	2	3	3	3	4	4	4	4
Last Grp. $(j..k)$	1-1	2-2	1-2	3-3	2-3	1-3	4-4	3-4	2-4	1-4
Prev. DBA $[j-1]$	0	1	0	2	1	0	3	2	1	0
Cost	1	3	2	4	3	3	6	5	5.5	6

component of our DBA algorithm.

Lemma 3: Let $\tau^F(t)' = (w_1, \dots, w_N)$ be a sequence of $N = |\tau^F(t)|$ active fine subtask workloads, sorted non-decreasingly by patch count. The contiguous batch partition that minimizes the total PET can be computed in $O(N^2)$ time using dynamic programming.

Proof: The problem exhibits optimal substructure. Let $DP[k]$ be the minimum total PET for partitioning the prefix $w_1, \dots, w_k$. The base case is $DP[0] = 0$. The recurrence relation is:

$$DP[k] = \min_{1 \leq j \leq k} \{DP[j-1] + C_{batch}(w_k, k-j+1)\},$$

where $C_{batch}(w_k, k-j+1)$ is the PET of the last batch $(w_j, \dots, w_k)$. Since the sequence is sorted by workload, w_k is the maximum workload in this last batch. Computing each $DP[k]$ requires $O(k)$ time, leading to a total time complexity of $O(N^2)$ and space complexity of $O(N)$. ■

Next, we extend this PET-optimal partitioning to incorporate real-time constraints, constituting the full DBA algorithm.

Lemma 4: The DBA algorithm finds a schedulable, contiguous batch partition of the sorted workload sequence $\tau^F(t)'$ by adapting the dynamic programming approach from Lemma 3. It evaluates the following recurrence:

$$DBA[k] = \min_{1 \leq j \leq k} \{\text{Cost}(j, k)\}, \quad (6)$$

where $\text{Cost}(j, k)$ is defined as $DBA[j-1] + C_{batch}(w_k, k-j+1)$ if the sub-partition up to batch $(w_j, \dots, w_k)$ is schedulable (respecting real-time constraints (i) and (ii)), and ∞ otherwise. This finds the minimum-cost schedulable partition in $O(N^2)$ time.

Proof: The proof follows the structure of Lemma 3. The key modification is the cost function, $\text{Cost}(j, k)$, used within the DP recurrence. By assigning an infinite cost to any sub-partition that violates the specified real-time constraints (i) and (ii), the algorithm effectively prunes any invalid scheduling paths from the search space. The minimization operation in Eq. (6) therefore ensures that $DBA[k]$ represents the minimum cost for a schedulable partition of the prefix $w_1, \dots, w_k$. The overall time and space complexities remain $O(N^2)$ and $O(N)$ respectively, as the constraint checks are assumed to be efficient ($O(1)$ on average) for each step. ■

Table I exemplifies the DBA algorithm's computation (Eq. (6), where terms can be ∞ if constraints are violated) for $|\tau^F(t)| = 4$ fine subtasks with sorted workloads $w = \{S, M, M, L\}$ (S=1, M=2, L=3). Batch PET $C_{batch}(w_k, \text{size})$ is w_k if size = 1, or $(w_k \times \text{size})/2$ if size > 1 (a 50%

reduction). The table details calculating $\text{DBA}[k]$ values. For instance, when computing $\text{DBA}[3]$ (for $w_3 = M = 2$), options include forming the last batch as $\{w_3\}$, $\{w_2, w_3\}$, or $\{w_1, w_2, w_3\}$. If an option (e.g., batch $\{w_1, w_2, w_3\}$ with $\text{DBA}[0]$) violates slack (constraint (ii)) or internal deadlines (constraint (i)), its $\text{Cost}(j, k)$ in Eq. (6) becomes ∞ . $\text{DBA}[3]$ is the minimum of such costs. If the final $\text{DBA}[4]$ is finite (e.g., 5, yielding partition SM — ML), backtracking finds a schedulable sequence for all four tasks. Algo. 3 (Line 4) then dispatches an executable prefix of this sequence (the full sequence if $\text{DBA}[4]$ allows). If $\text{DBA}[4] = \infty$, no full schedulable partition exists; a shorter valid prefix (from some $\text{DBA}[k < 4]$) or no fine batches might be executed, always ensuring adherence to timing guarantees.

E. Schedulability of the NPFP** Framework

Based on the foundational lemmas, we now establish the overall schedulability of any policy in the NPFP** framework.

Theorem 1: If a task set τ is schedulable under the baseline NPFP^C policy (per Lemma 1), then any policy instantiated from the NPFP** framework is also schedulable with respect to its safety-critical coarse subtasks.

Proof: The schedulability of the framework’s safety-critical coarse subtasks is fundamentally guaranteed. Per Lemma 1, the task set is schedulable for individual coarse execution, and Lemma 2 ensures that the optional coarse batching mechanism (IC) preserves this baseline schedulability.

The execution of any optional fine subtask (F or [F]) is architecturally prevented from compromising this guarantee. Fine subtasks possess lower priority and are only dispatched when no coarse subtasks are ready, within a temporal window that must conclude before the next coarse subtask’s release. The DBA mechanism, detailed in Lemmas 3 and 4, constructs schedulable fine batches that strictly adhere to these non-interfering constraints. Thus, as opportunistic fine-grained processing does not impact the guaranteed coarse subtask schedule, the overall framework remains schedulable. ■

V. EVALUATION

A. Experimental setup

Experimental environment. Experiments are conducted across three distinct hardware platforms: a server equipped with an NVIDIA A10 GPU, an NVIDIA Jetson Orin [17], and a 1/10 scale AV featuring a Jetson TX2 board, which are utilized for the case study. DINO, the transformer-based object detection model, operating at FP16 precision, is employed throughout these evaluations. Its coarse stage, incorporating a ResNet-50 backbone [23] and six encoder/decoder layers, is pre-trained on the COCO dataset [24] and subsequently fine-tuned on KITTI [16] and BDD100K [25]. These datasets are equally divided, with one half allocated for model training and fine-tuning, and the other reserved for assessing the performance of the CF-DETR pipeline and its scheduler.

Execution time profiling and run-time overhead. PETs of individual components for coarse subtasks (C_i^S) and fine

TABLE II
EXECUTION TIME MEASUREMENT (MS) FOR COMPONENTS

Platform	Component	C_i^S			C_i^F			
		c_i^{ps}	$c_i^{at}(p_i^S)$	c_i^{dt}	c_i^{sp}	$c_i^{at}(S)$	$c_i^{at}(M)$	$c_i^{at}(L)$
Server	Mean	20	45	0.2	2	44	52	55
	Max	30	49	0.3	3	46	55	58
Orin	Mean	59	68	0.5	5	65	80	85
	Max	70	69	0.7	8	78	96	107

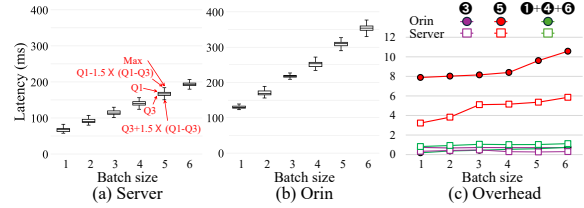


Fig. 6. Latency of coarse batch execution by size on (a) server and (b) Orin, and (c) average runtime overhead of CF-DETR’s components by batch size.

subtasks (C_i^F) are determined via 1000 measurements each on both the server and the Jetson Orin platform, with detailed results presented in Table II. Batches were composed by randomly selecting images from the KITTI dataset for every iteration to match the desired batch size; inference was then performed to measure latency. Figs. 6(a) and (b) illustrate the latency distributions for coarse batch sizes ($|B^S|$) ranging from one (no batch) to six on these platforms. Furthermore, Fig. 6(c) depicts the average run-time overhead of core operational components: hardness identification (cf. c_i^{dt} , ③ in Fig. 3), selective patch split (cf. c_i^{sp} , ⑤ in Fig. 3), and scheduling overhead (cf. Algo. 1, ①+④+⑥ in Fig. 3) for varying batch sizes. Specifically, the overhead for hardness identification (c_i^{dt}) and scheduling is minimal (under 1 ms). Selective patch split (c_i^{sp}) incurs approximately 8–11 ms, depending on the batch size, which is a reasonably small addition compared to the inference time ($c_i^{at}(p)$).

B. Experimental results

We evaluate the following four CF-DETR scheduling algorithms NPFP** on their effectiveness in improving accuracy (R2) while meeting real-time constraints (R1): (i) NPFP^C executing only individual coarse subtasks without batching, (ii) NPFP^{C[F]} with individual coarse subtasks and batched fine subtasks, (iii) NPFP^{C[F]} with batched coarse subtasks and individual fine subtasks, and (iv) NPFP^{C[F]} where both coarse and fine subtasks are batched.

We compare these against two DNN-SAM [7] scheduling algorithms, an existing approach for R1/R2 satisfaction in multi-MOD tasks: (v) DNN-SAM^{MF} (MandFirst) that favors mandatory subtask frames per second (FPS) by always prioritizing it over the optional subtask, and (vi) DNN-SAM^{EDF-S} (EDF-slack) that favors overall accuracy by adaptively prioritizing the optional subtask via runtime slack reclamation.

All six evaluated algorithms uniformly employ DINO [15] as the detector. For all evaluations, task sets satisfy offline schedulability tests: CF-DETR algorithms adhere to Lemma 1 under rate monotonic (RM) [26] priority assignment, while

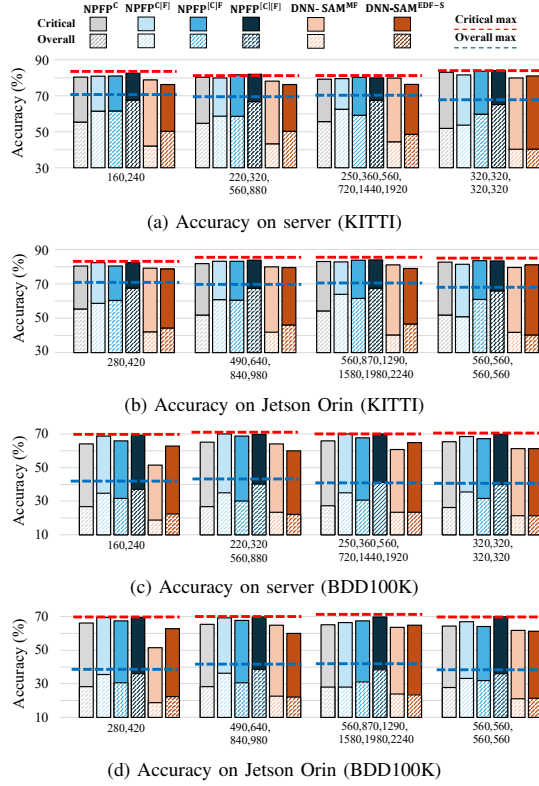


Fig. 7. Comparison of critical and overall mAP (%) between CF-DETR (with NPFP** variants) vs. DNN-SAM across various task sets. Results are shown for the KITTI dataset on (a) a server and (b) a Jetson Orin, and for BDD100K on (c) the server and (d) the Jetson Orin.

DNN-SAM algorithms follow Theorem 1 in [7] using earliest deadline first (EDF) [27]. Detection accuracy is measured by mean average precision (mAP) [24]; “critical accuracy” is for critical objects (area > 16384 pixels), and “overall accuracy” for all objects (including critical ones). “Critical max” and “overall max” denote DINO’s maximum achievable accuracy (without schedulability guarantees). Task response times are measured in FPS.

Fig. 7 presents critical and overall accuracy results on two platforms. Experiments use task sets of two, four, or six tasks with varying periods (detailed on the x-axis of Fig. 7), all satisfying Lemma 1, ensuring guaranteed coarse subtask execution while fine subtask processing remains opportunistic. The period sets were chosen to demonstrate the system operating at its schedulability limits, representing the tightest possible values derived from the coarse inference’s execution time. For critical accuracy, all CF-DETR variants achieve near-maximum performance, outperforming DNN-SAM algorithms; note that critical accuracies can vary slightly among CF-DETR variants as fine subtasks may also detect critical objects missed by their coarse counterparts. This superiority stems from CF-DETR’s low-latency coarse patch attention effectively detecting large, dispersed critical objects while minimizing focus on irrelevant background. In contrast, the model-agnostic DNN-SAM

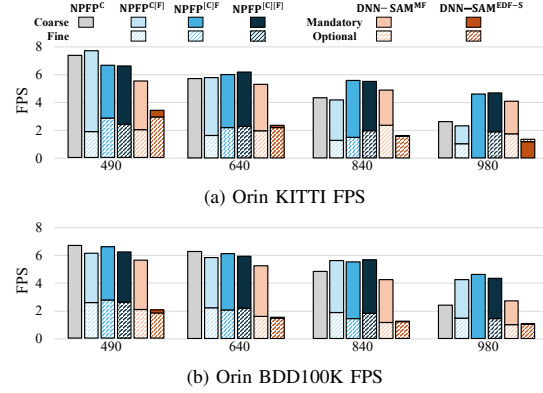


Fig. 8. Per-task FPS: CF-DETR (under NPFP** variants) vs. DNN-SAM on Jetson Orin for four-task sets (490, 640, 840, 980). The FPS for coarse and fine components is measured from the initial job release time of individual task to the completion of the corresponding respective subtask.

processes a single, rescaled, and potentially oversized region for all critical objects (including background), which can reduce critical accuracy. Regarding overall accuracy, DNN-SAM algorithms show a significant performance drop due to a lack of batch support and potentially requiring two full inferences if optional subtasks are invoked, without resource-effective adaptation. Conversely, CF-DETR variants, especially with multi-level batched execution, maintain higher overall accuracy. Compared to KITTI, the BDD100K dataset features a more heterogeneous distribution of large safety-critical objects and smaller non-critical objects. This characteristic led to a slight decrease in the critical accuracy for NPFP^C and DNN-SAM baselines; however, the overall performance trends remained largely consistent with those observed on the KITTI.

Fig. 8 shows per-task FPS results for representative task sets on the Jetson Orin platform, with tasks arranged by decreasing RM priority. Under CF-DETR’s policies, coarse subtasks are strictly prioritized, ensuring their execution is unimpeded by fine subtasks; this guarantees responsiveness for critical detections while enabling efficient fine-detail processing. Consequently, policies lacking efficient coarse subtask batching (e.g., NPFP^C and NPFP^{C[F]}), or those not prioritizing FPS like DNN-SAM^{EDF-S}, exhibit a marked FPS reduction for lower-priority tasks. Note that the FPS difference among tasks under NPFP^{C*} stems from fixed-priority scheduling, but is mitigated through coarse subtask batching in NPFP^{C[F]*}. As shown in Fig. 8, the system throughput demonstrates a consistent trend for both the KITTI and BDD100K datasets.

We empirically validate our choice of the confidence threshold for hardness identification in strategy A1, where an image is classified as “hard” if its average object query confidence exceeds this value. Fig. 9 illustrates the trade-off involved in this selection. As shown in Fig. 9(a), the average mAP gain per image processed by the fine stage—a metric indicating refinement efficiency—peaks at a threshold of 0.8 for both KITTI and BDD100K before showing diminishing returns. Meanwhile, Fig. 9(b) shows that the number of images sent to the fine stage, and thus the associated computational cost,

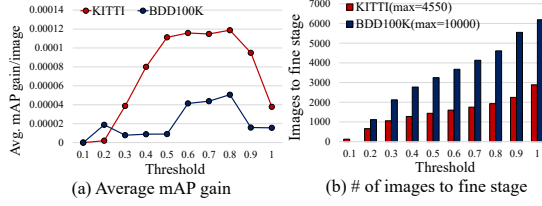


Fig. 9. Analysis of the hardness identification threshold in strategy A1 for the KITTI and BDD100K datasets: (a) average mAP gain per image vs. the confidence threshold, and (b) # of images passed to the fine-inference stage.

increases steadily with the threshold. Taken together, these results empirically demonstrate that a threshold of 0.8 maximizes the accuracy-latency trade-off by selectively processing only those images that benefit most from fine-grained refinement.

C. Case study: emergency braking

To validate the real-world practicality of CF-DETR, we conducted an emergency braking case study on a 1/10 scale AV equipped with a Jetson TX2 board. The vehicle, featuring two cameras (front and rear MOD tasks with periods of 1600 ms and 2400 ms, respectively), LiDAR, an IMU sensor, and multiple actuators, operated at 0.5–0.6 m/s speed and was programmed to stop upon detecting an object within 1.5 m. Execution times for CF-DETR components on the TX2 platform are detailed in Table III. We compared CF-DETR under the NPFP^[C|F] policy against the DINO baseline configured with a fine patch resolution by default to achieve its maximum possible accuracy, performing 10 trials with each approach.

The results, summarized in Fig. 10, highlight CF-DETR’s superior performance in ensuring safety. In all trials, DINO failed to detect the obstacle and stop the car before a collision, primarily due to the high perception latency incurred by its reliance on fine subtask processing for detection. In stark contrast, CF-DETR using the NPFP^[C|F] policy successfully avoided collisions in every trial. CF-DETR consistently brought the vehicle to a stop at a safe distance of 0.47–1.01 m from obstacles, with an average stopping distance of 0.88 m. This demonstrates the practical effectiveness of CF-DETR in safety-critical scenarios, achieving reliable obstacle avoidance while, as noted in the caption of Fig. 10, maintaining high overall accuracy comparable to that of DINO (cf. Fig. 7).

VI. RELATED WORK

Transformers, demonstrating superior accuracy over traditional CNNs/RNNs, have transitioned from NLP success to broad adoption in computer vision tasks like image classification [28] and object detection [3]. Their substantial computational cost and latency, however, have spurred research into lightweight architectures [4], [5], such as A-ViT [4] with token-halting and ToMe [5] with token merging. While effective in reducing average computational demands, these methods often lack reliable focus on safety-critical objects and regions required by safety-critical systems. Similarly, CF-ViT [13] introduced a coarse-to-fine strategy for classification

TABLE III
EXECUTION TIME MEASUREMENT (MS) FOR COMPONENTS

Platform	Component	C_i^S			C_i^F			
		c_i^{ps}	$c_i^{at}(p_i^S)$	c_i^{dt}	c_i^{ps}	$c_i^{at}(S)$	$c_i^{at}(M)$	$c_i^{at}(L)$
TX2	Mean	349	288	1.1	32	962	1207	1441
	WCET	408	368	1.5	38	1147	1245	1478

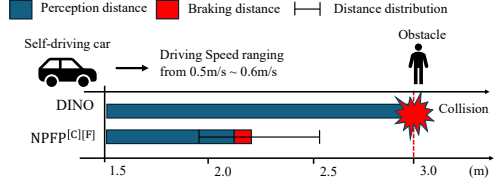


Fig. 10. Emergency braking: NPFP^[C|F] ensures safe stopping for critical objects, while achieving overall accuracy comparable to DINO (cf. Fig. 7).

by leveraging token size versus accuracy, but it does not address safety considerations or detection tasks.

Research in RT-DNN scheduling [6], [7] has targeted concurrent timely execution and accuracy satisfaction; approaches include DNN-SAM [7] prioritizing critical regions and RT-MOT [6] dynamically allocating resources for tracking, though often with suboptimal GPU utilization due to limited batching. Conversely, another line of work improved GPU utilization via batching [9]–[11] for responsiveness (e.g., in single multi-object detection tasks [9], S³DNN [10], DART [11]), but typically overlooked hard real-time constraints.

VII. CONCLUSION

In this paper, we proposed CF-DETR to overcome the limitations of existing generic RT-DNN approaches in concurrently executing multiple detection tasks for AV systems, addressing the crucial R1 & R2 balance challenge posed by DETR’s latency-accuracy trade-off. CF-DETR’s novelty lies in being the first to adapt the coarse-to-fine paradigm, previously explored in classification, to the complexities of object detection, thereby creating an architectural foundation (i.e., A1–A3) to effectively navigate this trade-off. Building upon this, the NPFP^{**} scheduler (i.e., A4) is specifically designed to achieve the overarching goals of R1 while achieving the high accuracy requirement R2. Extensive evaluations confirmed that CF-DETR consistently meets critical deadlines while significantly outperforming baselines in both critical and overall detection accuracy.

ACKNOWLEDGEMENT

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2023-00250742, RS-2024-00438248, RS-2025-00520023, RS-2022-NR068754, RS-2025-25424702). This research was supported by the MSIT, Korea under the ITRC support program(IITP-2025-RS-2023-00259061) supervised by the IITP. This work was supported by the IITP grant funded by the Korea government (MSIT) (RS-2024-00440780). This work was carried out as part of a collaborative research with New York University titled ‘Robust and private outsourced machine learning’.

REFERENCES

- [1] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat yolos on real-time object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 16 965–16 974.
- [2] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," *arXiv preprint arXiv:2010.04159*, 2020.
- [3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.
- [4] H. Yin, A. Vahdat, J. M. Alvarez, A. Mallya, J. Kautz, and P. Molchanov, "A-vit: Adaptive tokens for efficient vision transformer," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 809–10 818.
- [5] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, "Token merging: Your vit but faster," *arXiv preprint arXiv:2210.09461*, 2022.
- [6] D. Kang, S. Lee, H. S. Chwa, S.-H. Bae, C. M. Kang, J. Lee, and H. Baek, "RT-MOT: Confidence-aware real-time scheduling framework for multi-object tracking tasks," in *IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 318–330.
- [7] W. Kang, S. Chung, J. Y. Kim, Y. Lee, K. Lee, J. Lee, K. G. Shin, and H. S. Chwa, "DNN-SAM: Split-and-merge dnn execution for real-time object detection," in *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2022, pp. 160–172.
- [8] D. Kang, J. Lee, and H. Baek, "Real-time scheduling for multi-object tracking tasks in regions with different criticalities," *Journal of Systems Architecture*, p. 103349, 2025.
- [9] S. Liu, X. Fu, M. Wigness, P. David, S. Yao, L. Sha, and T. Abdelzaher, "Self-cueing real-time attention scheduling in criticality-aware visual machine perception," in *IEEE Real Time Technology and Applications Symposium (RTAS)*, 2022, pp. 173–186.
- [10] H. Zhou, S. Bateni, and C. Liu, "S³DNN: Supervised streaming and scheduling for GPU-accelerated real-time DNN workloads," in *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2018, pp. 190–201.
- [11] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *IEEE Real-Time Systems Symposium (RTSS)*, 2019, pp. 392–405.
- [12] W. Jang, H. Jeong, K. Kang, N. Dutt, and J.-C. Kim, "R-TOD: Real-time object detector with minimized end-to-end delay for autonomous driving," in *IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 1–14.
- [13] M. Chen, M. Lin, K. Li, Y. Shen, Y. Wu, F. Chao, and R. Ji, "Cf-vit: A general coarse-to-fine method for vision transformer," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 6, 2023, pp. 7042–7052.
- [14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [15] H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni, and H.-Y. Shum, "Dino: Detr with improved denoising anchor boxes for end-to-end object detection," *arXiv preprint arXiv:2203.03605*, 2022.
- [16] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [17] NVIDIA Orin Developer Kit. [Online]. Available: <https://www.nvidia.com/ko-kr/autonomous-machines/embedded-systems/jetson-orin/>.
- [18] D. Kang, S. Lee, C.-H. Hong, J. Lee, and H. Baek, "Batch-MOT: Batch-enabled real-time scheduling for multi-object tracking tasks," in *Proceedings of ACM & IEEE International Conference on Embedded Software (EMSOFT)*, 2024, pp. 1–12.
- [19] A. Burns, K. Tindell, and A. Wellings, "Effective analysis for engineering real-time fixed priority schedulers," *IEEE Transactions on Software Engineering (TSE)*, vol. 21, no. 5, pp. 475–480, 1995.
- [20] G. Yao, G. Buttazzo, and M. Bertogna, "Feasibility analysis under fixed priority scheduling with fixed preemption points," in *IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2010, pp. 71–80.
- [21] J.-J. Chen and G. von der Bruggen, "Non-preemptive and limited preemptive scheduling," *Lecture Note in TU Dortmund*, 2017. [Online]. Available: <https://ls12-www.cs.tu-dortmund.de/daes/media/documents/teaching/courses/rts/non-preemptive-scheduling.pdf>
- [22] L. George, N. Rivierre, and M. Spuri, "Preemptive and non-preemptive real-time uniprocessor scheduling," Inria, Tech. Rep., 1996.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [24] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer vision—ECCV 2014: 13th European conference, zurich, Switzerland, September 6–12, 2014, proceedings, part v 13*. Springer, 2014, pp. 740–755.
- [25] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.
- [26] J. Lehoczy, L. Sha, and Y. Ding, "The rate monotonic scheduling algorithm: Exact characterization and average case behavior," in *IEEE Real-Time Systems Symposium (RTSS)*, 1989, pp. 166–171.
- [27] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM (JACM)*, vol. 20, no. 1, pp. 46–61, 1973.
- [28] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.